Common Python Libraries for EDA and Hypothesis Testing

Introduction

Before you can build any chart or run any statistical check, you need the right tools. In Python, these tools come as **libraries** - pre-built collections of code that someone else has already written, tested, and packaged up so you don't have to build everything from scratch.

This guide covers five libraries that form the backbone of almost any data analysis project:

Library	Import Line	Main Job
`pandas`	`import pandas as pd`	Organizing and manipulating data in tables
`numpy`	`import numpy as np`	Fast number crunching and math operations
`matplotlib`	`import matplotlib as plt`	Building and customizing charts from scratch
`seaborn`	`import seaborn as sns`	Building polished statistical charts quickly
`scipy`	`import scipy as stats`	Running statistical tests and probability calculations

A quick note on that `as pd` / `as np` / `as plt` / `as sns` / `as stats` pattern: it's simply a **nickname (alias)**. Instead of typing the full library name every time you use it, Python lets you give it a short alias. These specific nicknames are so universally used that seeing `pd.something` or `sns.something` in code instantly tells any Python user which library is being used - it's a shared convention, not a technical requirement.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

1. pandas

What Is It?

Pandas is like an interactive spreadsheet engine built directly into Python. Just as you might use Excel or Google Sheets to organize rows and columns of data, filter them, or calculate totals, pandas lets you do all of that - but through code, which makes it faster, more repeatable, and capable of handling far larger datasets than a spreadsheet program typically can.

Key Concept / Terminology

- **DataFrame**: The main object in pandas - think of it as a single spreadsheet or table, with rows and named columns. Almost everything you do in pandas starts with loading your data into a DataFrame.
- **Series**: A single column of a DataFrame, pulled out on its own. If a DataFrame is the whole spreadsheet, a Series is just one column of it, like the `age` column by itself.
- **Index**: The row labels of a DataFrame - by default, these are just row numbers (0, 1, 2, ...), similar to the row numbers on the left side of a spreadsheet.
- **Missing values (NaN)**: Pandas' way of marking a blank or unknown cell - short for "Not a Number." It's how pandas flags data that wasn't recorded.

When and Why We Use It

Pandas is typically the very first tool you reach for in any data project. Common moments to use it include:

- **Loading data**: Bringing in a CSV file or Excel file so Python can work with it.
- **Inspecting data**: Getting a quick look at what your data contains, how many rows it has, and what types of values are in each column.
- **Cleaning data**: Handling missing values, removing duplicate rows, or fixing incorrectly labeled entries.
- **Summarizing data**: Calculating averages, totals, counts, or other quick summary numbers for any column.
- **Filtering and grouping**: Answering questions like "show me only customers from California" or "what's the average sales figure per region?"

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2. `numpy`

What Is It?

NumPy (short for "Numerical Python") is like a high-powered calculator built for handling huge lists of numbers all at once, instead of one number at a time. If pandas is your spreadsheet engine, numpy is the engine *underneath* it - doing the fast math that makes many pandas operations possible in the first place.

Key Concept / Terminology

- **Array**: NumPy's core object - essentially a list of numbers (or a grid of numbers) that NumPy can perform calculations on very quickly, all at once, instead of looping through them one by one.
- **Shape**: Describes the size and dimensions of an array - for example, a shape of `(100,)` means a simple list of 100 numbers, while `(100, 3)` means a table-like grid with 100 rows and 3 columns.
- **NaN**: Just like in pandas, this stands for "Not a Number" and represents a missing or undefined numeric value.

When and Why We Use It

NumPy is used whenever you need fast, reliable math on numbers - often behind the scenes, but sometimes directly. Typical moments include:

- **Bulk calculations**: Applying the same formula (like converting units, or calculating percentages) to an entire column of numbers at once.
- **Statistical building blocks**: Calculating basics like averages, standard deviation (a measure of spread), or rounding - which other libraries, including pandas and scipy, rely on internally.
- **Handling arrays directly**: Working with numeric data in its rawest form, especially useful for more advanced numerical or scientific calculations.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

3. matplotlib

What is it?

Matplotlib is the original, foundational charting library in Python. While Matplotlib is the overall library, we specifically use the `matplotlib.pyplot` module to create charts, where a module can be thought of as a specialized toolbox inside the larger library. It acts as a blank canvas and a full set of drawing tools that doesn't make assumptions about how your chart should look; instead, it gives you full control over every detail, from axis labels to colors to chart size. It's less like a template and more like a drafting table where you build the chart piece by piece.

Key Concept / Terminology

- **Figure**: The overall "canvas" or window that holds your entire chart (or multiple charts, if you're building several at once).
- **Axes**: The actual plotting area within a figure - where the bars, lines, or dots are drawn, including the x-axis and y-axis.
- **Plot**: A general term for any chart or graph - a line plot, bar plot, scatter plot, and so on.
- **Labels/Title**: The text elements that explain what a chart is showing - axis labels describe what each axis represents, and the title describes the chart as a whole.

When and Why We Use It

Matplotlib is used whenever you need to build or fine-tune a visual, especially when you want full control over its appearance. Common moments include:

- **Building basic charts**: Line charts, bar charts, histograms, and scatter plots from raw data.
- **Adding annotations**: Placing text or arrows directly on a chart to highlight specific data points or interesting trends.
- **Customizing appearance**: Adjusting titles, axis labels, colors, sizes, and layouts to make a chart presentation-ready.
- **Combining multiple charts**: Placing several charts side-by-side or in a grid within a single figure.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. seaborn

What Is It?

Seaborn is a higher-level charting library built directly on top of matplotlib - think of it as a set of ready-made, professionally designed chart templates. Where matplotlib gives you raw tools to build a chart from scratch, seaborn gives you polished, well-made charts in far fewer lines of code, with attractive defaults already built in.

Key Concept / Terminology

- **Statistical visualization**: Charts specifically designed to reveal patterns, comparisons, or relationships in data - not just decoration, but visuals built to help you *analyze*.
- **Hue**: Seaborn's term for coloring parts of a chart based on a category (for example, coloring dots differently depending on `gender` or `region`), which lets a single chart show an extra layer of information.
- **DataFrame-aware plotting**: Seaborn is built to work directly with pandas DataFrames, meaning you can usually refer to column names directly (like `x="age"`) rather than manually pulling out raw lists of numbers.

When and Why We Use It

Seaborn is typically used once you're ready to explore relationships and patterns more efficiently than matplotlib alone allows. Common moments include:

- **Quick statistical charts**: Building histograms, boxplots, scatter plots, and bar charts in just one or two lines of code.
- **Comparing groups**: Easily coloring or splitting charts by category to compare subgroups (e.g., comparing sales by region on the same chart).
- **Working directly with DataFrames**: Skipping manual data extraction steps by plotting straight from a pandas table.
- **Producing clean, presentation-ready visuals**: Seaborn's default styling is generally more polished out-of-the-box than matplotlib's, saving time on manual formatting.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

5. `scipy`

SciPy is a library focused on scientific and technical computing. While SciPy itself handles a huge range of engineering and math tasks, we primarily use the `scipy.stats` module for data analysis work. Think of `scipy.stats` as your specialized statistics toolbox that lives inside the larger SciPy library - it gives you all the pre-built commands you need to run formal statistical tests.

Key Concept / Terminology

- **Hypothesis Testing**: A structured method for checking whether an observed pattern (like "Campaign A had higher sales than Campaign B") is a genuine effect, or could easily have happened just by random chance.
- **Null Hypothesis**: The "default" assumption that there is *no real difference or effect* - for example, "there is no real difference in average sales between Campaign A and Campaign B." Hypothesis tests are designed to check whether the evidence is strong enough to doubt this default assumption.
- **p-value**: A number (between 0 and 1) that tells you how likely it would be to see a difference as large as the one observed, *if* the null hypothesis (no real difference) were actually true. A small p-value (commonly below 0.05) suggests the observed difference is unlikely to be due to chance alone, giving more confidence that a real effect exists.
- **Statistical significance**: A general term for when a p-value is small enough that a result is considered unlikely to be due to random chance alone, based on a threshold decided before the test (commonly 0.05).

When and Why We Use It

`scipy.stats` is used once you've spotted an interesting pattern through EDA and want to check whether it holds up to formal scrutiny. Common moments include:

- **Comparing two groups**: Checking whether an observed difference between two groups (like two marketing campaigns, or two customer segments) is likely to be real.
- **Checking relationships between categories**: Testing whether two categorical variables (like device type and purchase outcome) appear to be genuinely related.
- **Validating patterns before making decisions**: Moving beyond "it looks different in the chart" to a more rigorous "the evidence supports that it's genuinely different."

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How These Libraries Work Together

In a typical EDA and hypothesis testing project, these libraries are rarely used in isolation - they form a natural pipeline:

1. **`pandas`** loads and organizes your raw data into a clean, workable table.
2. **`numpy`** powers many of the underlying calculations, and can be used directly for quick numeric operations.
3. **`matplotlib.pyplot`** provides the foundational canvas for any visual, and offers fine-grained control when needed.
4. **`seaborn`** builds on matplotlib to quickly produce polished, insightful statistical charts straight from your pandas DataFrames.
5. **`scipy.stats`** takes the patterns you've spotted visually and puts them through a formal check, helping you decide whether they represent a real, meaningful effect.

Together, these five libraries cover the full journey from "loading a messy spreadsheet" to "confidently explaining a data-backed business decision."

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by sahithidonkina@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.